Option Pricing using Quantum Computers

Nikitas Stamatopoulos[1], Daniel J. Egger[2], Yue Sun[1], Christa Zoufal[2,3], Raban Iten[2,3], Ning Shen[1], and Stefan Woerner[2]

[1]Quantitative Research, JPMorgan Chase & Co., New York, NY, 10017

[2]IBM Quantum, IBM Research – Zurich

[3]ETH Zurich

arXiv:1905.02666v5 [quant-ph] 2 Jul 2020

We present a methodology to price options and portfolios of options on a gate-based quantum computer using amplitude estimation, an algorithm which provides a quadratic speedup compared to classical Monte Carlo methods. The options that we cover include vanilla options, multi-asset options and path-dependent options such as barrier options. We put an emphasis on the implementation of the quantum circuits required to build the input states and operators needed by amplitude estimation to price the different option types. Additionally, we show simulation results to highlight how the circuits that we implement price the different option contracts. Finally, we examine the performance of option pricing circuits on quantum hardware using the IBM Q Tokyo quantum device. We employ a simple, yet effective, error mitigation scheme that allows us to significantly reduce the errors arising from noisy two-qubit gates.

1 Introduction

Options are financial derivative contracts that give the buyer the right, but not the obligation, to buy (call option) or sell (put option) an underlying asset at an agreed-upon price (strike) and timeframe (exercise window). In their simplest form, the strike price is a fixed value and the timeframe is a single point in time, but exotic variants may be defined on more than one underlying asset, the strike price can be a function of several market parameters and could allow for multiple exercise dates. As well as providing investors with a vehicle to profit by taking a view on the market or exploit arbitrage opportunities, options are core to various hedging strategies and as such, understanding their properties is a fundamental objective of financial engineering. For an overview of option types, features and uses, we refer the reader to Ref. [1].

Due to the stochastic nature of the parameters options are defined on, calculating their fair value can be an arduous task and while analytical models exist for the simplest types of options [2], the simplifying assumptions on the market dynamics required for the models to provide closed-form solutions often limit their applicability [3]. Hence, more often than not, numerical methods have to be employed for option pricing, with Monte Carlo being one of the most popular due to its flexibility and ability to generically handle stochastic parameters [4, 5]. However, despite their attractive features in option pricing, classical Monte Carlo methods generally require extensive computational resources to provide accurate option price estimates, particularly for complex options. Because of the widespread use of options in the finance industry, accelerating their convergence can have a significant impact in the operations of a financial institution.

By leveraging the laws of quantum mechanics a quantum computer [6] may provide novel ways to solve computationally intensive problems such as quantum chemistry [7–10], solving linear systems of equations [11], and machine learning [12–14]. Quantitative finance, a field with many computationally hard problems, may benefit from quantum computing. Recently developed applications of gate-based quantum computing for use in finance [15] include portfolio optimization [16], the calculation of risk measures [17] and pricing derivatives [18–20]. Several of these applications are based on the Amplitude Estimation algorithm [21] which can estimate a parameter with a convergence rate of $1/M$, where M is the number of quantum samples used. This represents a theoretical quadratic speed-up compared to classical Monte Carlo methods.

In this paper we extend the pricing methodology presented in [17, 18] and place a strong emphasis on the implementation of the algorithms in a gate-based quantum computer. We first classify options according to their features and show how to take the different features into account in a quantum computing setting. In Sec. 3, we review the quantum methodology to price options and discuss how to represent relevant probability distributions in a quantum computer. In Sec. 4, we show a framework to price vanilla options and portfolios of vanilla options, options with path-dependent dynamics and options on several underlying assets. In Sec. 5 we show results from eval-

Nikitas Stamatopoulos: Current Address: Goldman Sachs & Co., New York, NY, 10282

uating our option circuits on quantum hardware, and describe the error mitigation scheme we employ to increase the accuracy of the estimated option prices. In particular, we employ the maximum likelihood estimation method introduced in [22] to perform amplitude estimation without phase estimation in option pricing using three qubits of a real quantum device.

2 Review of option types and their challenges

Option contracts are valid for a pre-determined period of time, and their value at the expiration date is called the *payoff*. The goal of option pricing is to estimate the option payoff at the expiration date in the future and then *discount* that value to determine its worth today. The discounted payoff is also called the *fair value* and indicates the amount of money one should pay to enter the option contract today, making it worthwile receiving the payoff value at the expiration date.

In practice, we price complex options numerically using Monte Carlo methods by following these steps:

1. Model the asset price of the option's underlying(s) and any other sources of uncertainty as random variables $\mathbf{X} = \{X_1, X_2, \ldots, X_N\}$ following a stochastic process.

2. Generate a large number M of random price paths $\{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_M\}$ for the underlying(s) from the probability distribution $\mathbb{P}$ implied by the stochastic process.

3. Calculate the option's payoff $f(\mathbf{X}_i)$ on each generated price path and compute an estimator for the expectation value of the payoff $\mathbb{E}_{\mathbb{P}}[f(\mathbf{X})]$ as an average across all paths

$$\hat{\mathbb{E}}_{\mathbb{P}}[f(\mathbf{X})] = \frac{1}{M}\sum_{i=1}^{M} f(\mathbf{X}_i)$$

   By the Central Limit Theorem, the estimator $\hat{\mathbb{E}}_{\mathbb{P}}$ converges to the expectation value $\mathbb{E}_{\mathbb{P}}$ as the number of paths goes to infinity, with convergence $\mathcal{O}\left(M^{-1/2}\right)$ [23].

4. Discount the calculated expectation value to get the option's fair value.

The discounting process requires knowledge of interest rates at future dates which is itself an important question from a financial modelling perspective. However, for the types of options we consider, this process is not computationally challenging and can be performed classically after the payoff calculation. We therefore do not discount the expected payoff for simplicity.

We classify options according to two categories: path-independent vs path-dependent and options on a single asset or on multiple assets. Path-independent options have a payoff function that depends on an underlying asset at a single point in time. Therefore, the price of the asset up to the exercise date of the option is irrelevant for the option price. By contrast, the payoff of path-dependent options depends on the evolution of the price of the asset and its history up to the exercise date. Table 1 exemplifies this classification. Options that are path-independent and rely on a single asset are the easiest to price, and in most cases numerical calculation is straightforward and would likely not benefit by the use of a quantum computer. Path-independent options on multiple assets are only slightly harder to price since more than one asset is now involved and the probability distributions must account for correlations between the assets, but usually these can be priced quite efficiently on classical computers as well. Path-dependent options on the other hand are significantly harder to price than path-independent options since they require an often expensive payoff calculation at multiple time points on each path, therefore minimizing the number of paths required for this step would lead to a significant benefit in the pricing process. It is this last case where we envision the largest impact of quantum computing.

3 Quantum Methodology

Here we outline the building blocks needed to price options on a gate-based quantum computer. As discussed in the previous section, the critical components are 1) represent the probability distribution $\mathbb{P}$ describing the evolution of random variables $\mathbf{X} = \{X_1, X_2, \ldots, X_N\}$ on the quantum computer, 2) construct the circuit which computes the payoff $f(\mathbf{X})$ and 3) calculate the expectation value of the payoff $\mathbb{E}_{\mathbb{P}}[f(\mathbf{X})]$. In Sec. 3.1 we show how to use Amplitude Estimation to calculate the expectation value of a function of random variables. In Sec. 3.2 we describe the process of loading the relevant probability distributions to a quantum register, and in Sec. 3.3 we construct the circuits to compute the payoff and set up Amplitude Estimation to estimate the expectation value of the payoff. We then have all the ingredients to price options on a quantum computer.

3.1 Amplitude Estimation

The advantage of pricing options on a quantum computer comes from the Amplitude Estimation (AE) algorithm [21] which provides a quadratic speed-up over classical Monte Carlo simulations [24, 25]. Suppose a unitary operator $\mathcal{A}$ acting on a register of $(n+1)$ qubits such that

$$\mathcal{A}\left|0\right\rangle_{n+1} = \sqrt{1-a}\left|\psi_0\right\rangle_n\left|0\right\rangle + \sqrt{a}\left|\psi_1\right\rangle_n\left|1\right\rangle \qquad (1)$$

Table 1: Example of the different option types.

	Single-asset	Multi-asset
Path-independent	European put/call	Basket option
Path-dependent	Barrier & Asian options	Multi-asset barrier options

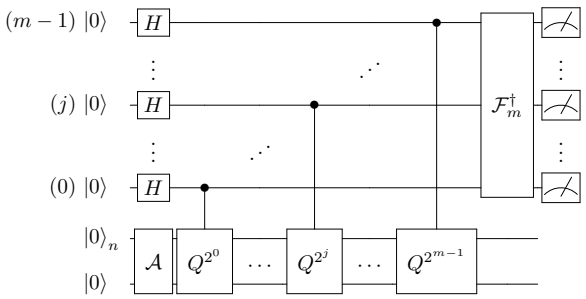

Figure 1: The quantum circuit of amplitude estimation, where H denotes a Hadamard gate and $\mathcal{F}^\dagger$ the inverse QFT.

for some normalized states $|\psi_0\rangle_n$ and $|\psi_1\rangle_n$, where $a \in [0,1]$ is unknown. AE allows the efficient estimation of a, i.e., the probability of measuring $|1\rangle$ in the last qubit. This estimation is obtained with an operator $\mathcal{Q} = \mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_{\psi_0}$, where $\mathcal{S}_0 = 1 - 2\,|0\rangle\langle 0|$ and $\mathcal{S}_{\psi_0} = 1 - 2\,|\psi_0\rangle\,|0\rangle\langle\psi_0|\langle 0|$, which is a rotation of angle $2\theta_a$ in the two-dimensional space spanned by $|\psi_0\rangle_n|0\rangle$ and $|\psi_1\rangle_n|1\rangle$. From Eq. (1) we know that $a = \sin^2(\theta_a)$. To obtain an approximation for θ_a, AE applies Quantum Phase Estimation [26, 27] to approximate certain eigenvalues of $\mathcal{Q}$, which are classically mapped to an estimator for a. The Quantum Phase Estimation uses m additional sampling qubits to represent result and $M = 2^m$ applications of $\mathcal{Q}$, i.e., M quantum samples. The m qubits, initialized to an equal superposition state by Hadamard gates, are used to control different powers of $\mathcal{Q}$. After applying an inverse Quantum Fourier Transform (QFT), their state is measured resulting in an integer $y \in \{0, ..., M-1\}$, which is classically mapped to the estimator for a, i.e.

$$\tilde{a} = \sin^2(y\pi/M) \in [0,1]. \tag{2}$$

The full circuit for AE is shown in Fig. 1. The estimator $\tilde{a}$ satisfies

$$|a - \tilde{a}| \leq \tfrac{\pi}{M} + \tfrac{\pi^2}{M^2} \;=\; \mathcal{O}\left(M^{-1}\right), \tag{3}$$

with probability of at least $8/\pi^2$. This represents a quadratic speedup compared to the $\mathcal{O}\left(M^{-1/2}\right)$ convergence rate of classical Monte Carlo methods [28].

To reduce the required number of qubits and the resulting circuit depth, Suzuki *et al.* have shown that AE can be performed without requiring quantum phase estimation while still maintaining a quadratic speed-up [22]. To this extent, they exploit that

$$\mathcal{Q}^k\mathcal{A}\,|0\rangle_n|0\rangle = \cos\left((2k+1)\theta_a\right)|\psi_0\rangle_n|0\rangle + \sin\left((2k+1)\theta_a\right)|\psi_1\rangle_n|1\rangle\,, \tag{4}$$

and by measuring $\mathcal{Q}^k\mathcal{A}\,|0\rangle$ for $k = 2^0, ..., 2^{m-1}$ for a given m and applying a maximum likelihood estimation, an approximation for θ_a (and hence a) can be recovered. If we define $M = 2^m - 1$, i.e. the total number of $\mathcal{Q}$-applications, and we consider N shots for each experiment, it has been shown empirically that the resulting estimation error scales as $\mathcal{O}(1/(M\sqrt{N}))$, i.e., the algorithm achieves the quadratic speed-up in terms of M. We will use this approach to demontrate results from real quantum hardware in Sec. 5.

For the option contracts we consider, the random variables involved represent the possible values S_i the underlying asset can take, and the corresponding probabilities p_i that those values will be realized. For an option with payoff f, the $\mathcal{A}$ operator will create the state

$$\sum_{i=0}^{2^n-1}\sqrt{1-f(S_i)}\sqrt{p_i}\,|S_i\rangle\,|0\rangle + \sum_{i=0}^{2^n-1}\sqrt{f(S_i)}\sqrt{p_i}\,|S_i\rangle\,|1\rangle\,. \tag{5}$$

Comparing Eq. (1) and Eq. (5), we can see that

$$a = \sum_{i=0}^{2^n-1} f(S_i)p_i = \mathbb{E}[f(S)], \tag{6}$$

meaning AE allows us to compute the undiscounted price of an option given a way to represent the option's payoff as a quantum circuit and create the state of Eq. (5). In the following sections, we describe the necessary components to achieve that.

3.2 Distribution loading

The first component of our option pricing model is the circuit that takes a probability distribution implied for possible asset prices in the future and loads it into a quantum register such that each basis state represents a possible value and its amplitude the corresponding probability. In other words, given an n-qubit register, asset prices $\{S_i\}$ for $i \in \{0, ..., 2^n - 1\}$ and corresponding probabilities $\{p_i\}$, the distribution loading module creates the state:

$$|\psi\rangle_n = \sum_{i=0}^{2^n-1} \sqrt{p_i}\,|S_i\rangle_n\,. \tag{7}$$

The analytical formulas used to price options in the Black-Scholes-Merton (BSM) model [2, 29] assume that the underlying stock prices at maturity follow a log-normal distribution with constant volatility. In [30], the authors show that log-concave probability distributions (such as the log-normal distribution of the BSM model) can be efficiently loaded in a gate-based quantum computer. The option types considered in this paper are modeled using the underlying BSM dynamics and thus loading the relevant probability distributions onto quantum registers does not require prohibitive complexity.

However, in option models of practical interest, the simplified assumptions in the BSM model fail to capture important market dynamics, limiting the model's applicability in real-life scenarios. As such, the market-implied probability distribution of the underlying needs to be captured properly in order for valuation models to accurately estimate the intrinsic value of option contracts. To address these shortcomings, Artificial Neural Networks (ANN) are becoming increasingly more popular as a means to capture the real-life dynamics of the relevant market parameters, without the need to assume a simplified underlying model [31, 32]. It is thus important to be able to efficiently represent distributions of financial parameters on a quantum computer which might not have explicit analytical representations.

The loading of arbitrary states into quantum systems requires exponentially many gates [33], making it inefficient to model arbitrary distributions as quantum gates. Since the distributions of interest are often of a special form, the limitation may be overcome by using quantum Generative Adverserial Networks (qGAN). These networks allow us to load a distribution using a polynomial number of gates [19]. A qGAN can learn the random distribution X underlying the observed data samples $\{\,x^0,\ldots,x^{k-1}\,\}$ and load it directly into a quantum state. This generative model employs the interplay of a classical discriminator, a neural network [34], and a quantum generator (a parametrized quantum circuit). More specifically, the qGAN training consists of alternating optimization steps of the discriminator's parameters ϕ and the generator's parameters θ. After the training, the output of the generator is a quantum state

$$|\psi(\theta)\rangle_n = \sum_{i=0}^{2^n-1} \sqrt{p_i(\theta)}\,|i\rangle_n\,, \tag{8}$$

that represents the target distribution. The n-qubit state $|i\rangle_n = |i_{n-1}...i_0\rangle$ encodes the integer $i = 2^{n-1}i_{n-1}+...+2i_1+i_0 \in \{0,...,2^n-1\}$ with $i_k \in \{0,1\}$ and $k = 0,...,n-1$. The probabilities $p_i(\theta)$ approximate the random distribution underlying the training data. We note that the outcomes of a random variable X can be mapped to the integer set $\{0,...,2^n-1\}$ using an affine mapping. Furthermore, the approach can be easily extended to multivariate data, where we use a separate register of qubits for each dimension [19].

Another useful feature in the use of qGANs for loading probability distributions, is the fact that we can tailor the qGAN variational form to construct short-depth circuits for an acceptable degree of accuracy. This in turn allows us to evaluate the performance of option pricing quantum circuits in near-term quantum hardware where resources are still quite limited.

The use of ANNs to represent probability distributions inevitably imposes a cost associated with the training component, in both classical and quantum models. However, during common business practices such as overnight risk assessment of large portfolios which may consist of millions of option contracts, the same probability distributions can be used across a large number of pricing calls defined on the same underlyings. For example, it is quite common during risk assessment valuations to require pricing of several thousands of option contracts defined on the same underlying. As such, the effective training cost per pricing call can be efficiently amortized to represent only a small fraction of the total individual option pricing cost.

It is also noteworthy that the qGAN training performs better if the initial distribution is close to the target distribution [19]. Therefore, as new market data comes in which needs to be incorporated into the probability distribution (e.g. for overnight risk, a lot of the same options as the day before need to be priced, but there is one more day's worth of market data) the previously trained qGAN can be used as the initial distribution, leading to faster convergence.

3.3 Computing the payoff

We obtain the expectation value of a linear function f of a random variable X with AE by creating the operator $\mathcal{A}$ such that $a = \mathbb{E}[f(X)]$, see Eq. (6). Once $\mathcal{A}$ is implemented we can prepare the state in Eq. (1), and the $\mathcal{Q}$ operator, which only requires $\mathcal{A}$ and generic quantum operations [26, 27]. In this section, we show how to create a close relative of $\mathcal{A}$ and how to combine it with AE to calculate the expectation value of f.

The payoff function for the option contracts of interest is piece-wise linear and as such we only need to consider linear functions $f : \{0,...,2^n-1\} \to [0,1]$ which we write $f(i) = f_1 i + f_0$. We can efficiently create an operator that performs

$$|i\rangle_n\,|0\rangle \to |i\rangle_n\,(\cos[f(i)]\,|0\rangle + \sin[f(i)]\,|1\rangle) \tag{9}$$

using controlled Y-rotations [17]. To implement the linear term of $f(i)$ each qubit j (where $j \in \{0,\ldots n-$

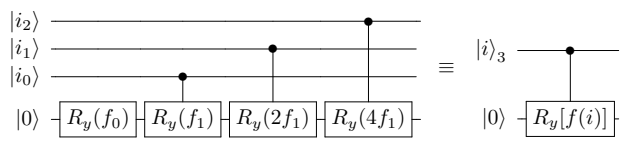

Figure 2: Quantum circuit that creates the state in Eq. (9). Here, the independent variable $i = 4i_2 + 2i_1 + i_0 \in \{0, ..., 7\}$ is encoded by three qubits in the state $|i\rangle_3 = |i_2 i_1 i_0\rangle$ with $i_k \in \{0, 1\}$. Therefore, the linear function $f(i) = f_1 i + f_0$ is given by $4f_1 i_2 + 2f_1 i_1 + f_1 i_0 + f_0$. After applying this circuit the quantum state is $|i\rangle_3 [\cos(f_1 i + f_0) |0\rangle + \sin(f_1 i + f_0) |1\rangle]$. The circuit on the right shows an abbreviated notation.

1}) in the $|i\rangle_n$ register acts as a control for a Y-rotation with angle $2^j f_1$ of the ancilla qubit. The constant term f_0 is implemented by a rotation of the ancilla qubit without any controls, see Fig. 2. The controlled Y-rotations can be implemented with CNOT and single-qubit gates [35].

We now describe how to obtain $\mathbb{E}[f(X)]$ for a linear function f of a random variable X which is mapped to integer values $i \in \{0, ..., 2^n - 1\}$ that occur with probability p_i respectively. To do this we use the procedure outlined immediately above to create the operator that maps $\sum_i \sqrt{p_i} |i\rangle_n |0\rangle$ to

$$\sum_{i=0}^{2^n-1} \sqrt{p_i} |i\rangle_n \left[\cos\left(c\tilde{f}(i) + \frac{\pi}{4}\right) |0\rangle + \sin\left(c\tilde{f}(i) + \frac{\pi}{4}\right) |1\rangle\right], \tag{10}$$

where $\tilde{f}(i)$ is a scaled version of $f(i)$ given by

$$\tilde{f}(i) = 2\frac{f(i) - f_{\min}}{f_{\max} - f_{\min}} - 1, \tag{11}$$

with $f_{\min} = \min_i f(i)$ and $f_{\max} = \max_i f(i)$, and $c \in [0, 1]$ is an additional scaling parameter. The relation in Eq. (11) is chosen so that $\tilde{f}(i) \in [-1, 1]$. Consequently, $\sin^2[c\tilde{f}(i) + \pi/4]$ is an anti-symmetric function around $\tilde{f}(i) = 0$. With these definitions, the probability to find the ancilla qubit in state $|1\rangle$, namely

$$P_1 = \sum_{i=0}^{2^n-1} p_i \sin^2\left(c\tilde{f}(i) + \frac{\pi}{4}\right),$$

is well approximated by

$$P_1 \approx \sum_{i=0}^{2^n-1} p_i \left(c\tilde{f}(i) + \frac{1}{2}\right) = c\frac{2\mathbb{E}[f(X)] - f_{\min}}{f_{\max} - f_{\min}} - c + \frac{1}{2}. \tag{12}$$

To obtain this result we made use of the approximation

$$\sin^2\left(c\tilde{f}(i) + \frac{\pi}{4}\right) = c\tilde{f}(i) + \frac{1}{2} + \mathcal{O}(c^3 \tilde{f}^3(i)) \tag{13}$$

which is valid for small values of $c\tilde{f}(i)$. With this first order approximation the convergence rate of AE is $\mathcal{O}(M^{-2/3})$ when c is properly chosen which is already faster than classical Monte Carlo methods [17]. We can recover the $\mathcal{O}(M^{-1})$ convergence rate of AE by using higher orders implemented with quantum arithmetic. The resulting circuits, however, have more gates. This trade-off, discussed in Ref. [17], also gives a formula that specifies which value of c to use to minimize the estimation error made when using AE. From Eq. (12) we can recover $\mathbb{E}[f(X)]$ since AE allows us to efficiently retrieve P_1 and because we know the values of $f_{\min}$, $f_{\max}$ and c.

4 Option pricing on a quantum computer

In this section we show how to price the different options shown in Tab. 1. We put an emphasis on the implementation of the quantum circuits that prepare the states needed by AE. We use the different building blocks reviewed in Sec. 3.

4.1 Path-independent options

The price of path-independent *vanilla* options (e.g. European call and put options) depends only on the distribution of the underlying asset price S_T at the option maturity T and the payoff function $f(S_T)$ of the option. To encode the distribution of S_T in a quantum state, we truncate it to the range $[S_{T,\min}, S_{T,\max}]$ and discretize this interval to $\{0, ..., 2^n - 1\}$ using n qubits. In the quantum computer, the distribution loading operator $\mathcal{P}_X$ creates a state

$$|0\rangle_n \xrightarrow{\mathcal{P}_X} |\psi\rangle_n = \sum_{i=0}^{2^n-1} \sqrt{p_i} |i\rangle_n, \tag{14}$$

with $i \in \{0, ..., 2^n - 1\}$ to represent S_T. This state, exemplified in Fig. 3, may be created using the methods discussed in Sec. 3.2.

We start by showing how to price vanilla call or put options, and then generalize our method to capture the payoff structure of portfolios containing more than one vanilla option.

4.1.1 Vanilla options

Vanilla call options are structured so that if the underlying asset price is larger than a fixed value K (the strike price) at the expiration date, the contract pays the difference between the realized price and the strike. As such, the call option payoff $f_C(S_T)$ can be written as

$$f_C(S_T) = \max(0, S_T - K). \tag{15}$$

Equivalently, the corresponding put option has a similar payoff but it pays if the asset price at expiry is

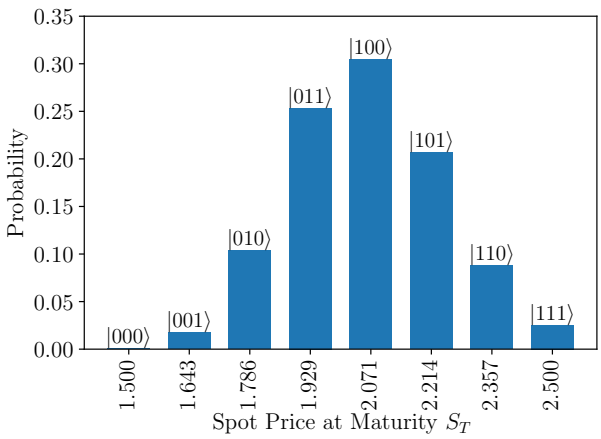

Figure 3: Example price distribution at maturity loaded in a three-qubit register. In this example we followed the Black-Scholes-Merton model which implies a log-normal distribution of the asset price at maturity T with probability density function $P(S_T) = \frac{1}{S_T\sigma\sqrt{2\pi T}}\exp\left(-\frac{(\ln S_T-\mu)^2}{2\sigma^2 T}\right)$. σ is the volatility of the asset and $\mu = \left(r - 0.5\sigma^2\right)T + \ln(S_0)$, with r the risk-free market rate and S_0 the asset's spot at $t = 0$. In this figure we used $S_0 = 2$, $\sigma = 10\%$, $r = 4\%$ and $T = 300/365$.

smaller than the strike. That is, the put option payoff is

$$f_P(S_T) = \max(0, K - S_T). \tag{16}$$

The linear part of the payoff can be computed as a quantum circuit with the approach outlined in Sec. 3.3, but we need a way to express the max operation as a quantum circuit as well. We show how to achieve that for both call and put option types by implementing a comparison between the values encoded in the basis states of Eq. (14) and K.

A quantum comparator circuit sets an ancilla qubit $|c\rangle$, initially in state $|0\rangle$, to the state $|1\rangle$ if $i \geq K$ and $|0\rangle$ otherwise. The state $|\psi\rangle_n$ in the quantum computer therefore undergoes the transformation

$$|\psi\rangle_n |0\rangle \to |\phi_1\rangle = \sum_{i<K} \sqrt{p_i}\, |i\rangle_n |0\rangle + \sum_{i\geq K} \sqrt{p_i}\, |i\rangle_n |1\rangle .$$

This operation can be implemented by a quantum comparator [36] based on CNOT and Toffoli gates. Since we know the value of the strike, we can implement a circuit tailored to the specific strike price. We use n ancilla qubits $|a_1, ..., a_n\rangle$ and compute the two's complement of the strike price K in binary using n bits, storing the digits in a (classical) array $t[n]$. For each qubit $|i_k\rangle$ in the $|i\rangle_n$ register, with $k \in \{0, ..., n-1\}$, we compute the possible carry bit of the bitwise addition of $|i_k\rangle$ and $t[k]$ into $|a_k\rangle$. If $t[k] = 0$, there is a carry qubit at position k only if there is a carry at position $k-1$ *and* $|i_k\rangle = 1$. If $t[k] = 1$, there is a carry qubit at position k if there is a carry at position $k-1$ *or* $|i_k\rangle = 1$. After going through all n qubits from least to most significant, $|i\rangle_n$ will be greater or equal to the strike price, only if there is a carry at the last (most significant) qubit. This procedure along with the necessary gate operations is illustrated in Fig. 4. An implementation for $K = 1.9$ and a three-qubit register is shown in Fig. 6.

To represent the payoff function $f(i)$, we add to $|\phi_1\rangle$ a second ancilla qubit, which corresponds to the last qubit in Eq. (10). The payoff function of vanilla options is piece-wise linear

$$f(i) = \begin{cases} a_< \cdot i + b_< & i < K, \\ a_\geq \cdot i + b_\geq & i \geq K. \end{cases} \tag{17}$$

We now focus on a European call option with payoff $f(i) = \max(0, i - K)$, i.e., $a_< = b_< = 0$, $a_\geq = 1$, and $b_\geq = -K$. To prepare the operator that calculates the payoff in the form of Eq. (10) for use with AE, we set

$$c\tilde{f}(i) + \frac{\pi}{4} = \begin{cases} g_0 & i < K, \\ g_0 + g(i) & i \geq K, \end{cases}$$

where $g(i)$ is a linear function of i, and g_0 is an angle whose value we will carefully select. With this setup, the payoff in Eq. (10) can be constructed, starting from the state $|\phi_1\rangle |0\rangle$, by first initializing the last ancilla qubit to the state $\cos(g_0)|0\rangle + \sin(g_0)|1\rangle$, and then performing a rotation of the last ancilla qubit controlled by the comparator qubit $|c\rangle$ and the qubits in $|\psi\rangle_n$. This rotation operation, implemented by the quantum circuit in Fig. 7, applies a rotation with an angle $g(i)$ only if $i \geq K$. The state of the $n+2$ qubits after this operation becomes

$$\sum_{i<K} \sqrt{p_i}\, |i\rangle_n |0\rangle \left[\cos(g_0)|0\rangle + \sin(g_0)|1\rangle\right] + \tag{18}$$
$$\sum_{i\geq K} \sqrt{p_i}\, |i\rangle_n |1\rangle \left\{\cos[g_0 + g(i)]|0\rangle + \sin[g_0 + g(i)]|1\rangle\right\}.$$

The probability to find the second ancilla in state $|1\rangle$, efficiently measurable using AE, is

$$P_1 = \sum_{i<K} p_i \sin^2(g_0) + \sum_{i\geq K} p_i \sin^2[g_0 + g(i)]. \tag{19}$$

Now, we must carefully choose the angle g_0 and the function $g(i)$ to recover the expected payoff $\mathbb{E}[f(X)]$ of the option from P_1 using the approximation in Eq. (12). To reproduce $f(i) = i - K$ for $i \geq K$ and simultaneously satisfy $c\tilde{f}(i) = g_0 + g(i) - \pi/4 \in [-c, c]$ (see Eq. (11)), we must set

$$g(i) = \frac{2c(i-K)}{i_{\max} - K}, \tag{20}$$

where $i_{\max} = 2^n - 1$. This choice of $g(i)$ forces us to choose

$$g_0 = \frac{\pi}{4} - c. \tag{21}$$

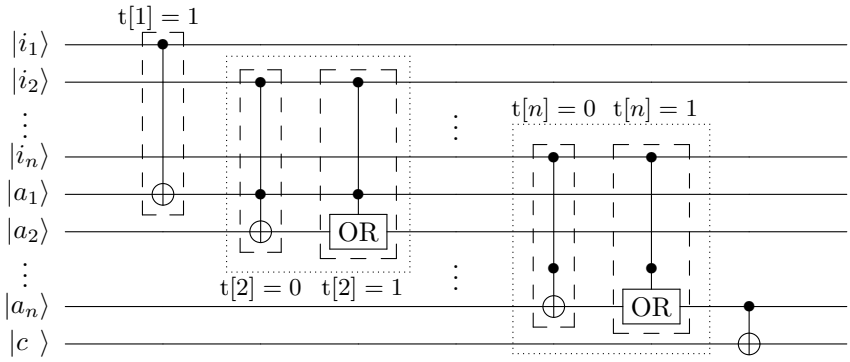

Figure 4: Circuit that compares the value represented by an n-qubit register $|i\rangle_n$, to a fixed value K. We use n ancilla qubits $|a_1, ..., a_n\rangle$, a classical array $t[n]$ holding the precomputed binary value of K's two's complement and a qubit $|c\rangle$ which will hold the result of the comparison with $|c\rangle = 1$ if $|i\rangle \geq K$. For each qubit $|i_k\rangle$, with $k \in \{1, ..., n\}$, we use a Toffoli gate to compute the carry at position k if $t[k] = 1$ and a logical OR, see Fig. 5, if $t[k] = 0$. For $k = 1$, we only need to use a CNOT on $|i_1\rangle$ if $t[1] = 1$. In the circuit above, only one of two unitaries in a dotted box needs to be added to the circuit, depending on the value of $t[k]$ at each qubit. The last carry qubit $|a_n\rangle$ is then used to compute the final result of the comparison in qubit $|c\rangle$.

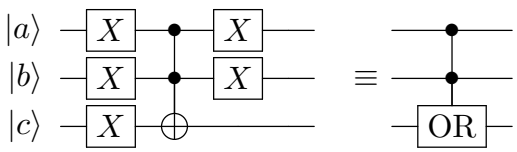

Figure 5: Circuit that computes the logical OR between qubits $|a\rangle$ and $|b\rangle$ into qubit $|c\rangle$. The circuit on the right shows the abbreviated notation used in Fig. 4.

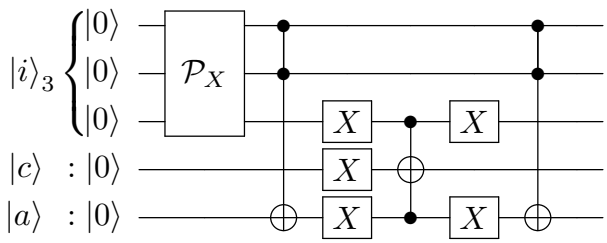

Figure 6: Quantum circuit that sets a comparator qubit $|c\rangle$ to $|1\rangle$ if the value represented by $|i\rangle_3$ is larger than a strike $K = 1.9$, for the spot distribution in Fig. 3. The unitary $\mathcal{P}_X$ represents the set of gates that load the probability distribution in Eq. (14). An ancilla qubit $|a\rangle$ is needed to perform the comparison. It is uncomputed at the end of the circuit.

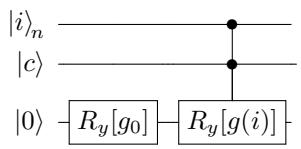

Figure 7: Circuit that creates the state in Eq. (18). We apply this circuit directly after the comparator circuit shown in Fig. 6. The multi-controlled y-rotation is the gate shown in Fig. 2 controlled by the ancilla qubit $|c\rangle$ that contains the result of the comparison between i and K.

To see why, we substitute Eqs. (20) and (21) into Eq. (19) and use the approximation in Eq. (13), which leads to

$$\begin{aligned} P_1 &\approx \sum_{i<K} p_i \left(\frac{1}{2} - c\right) + \sum_{i\geq K} p_i \left(\frac{2c(i-K)}{i_{\max} - K} + \frac{1}{2} - c\right) \\ &= \frac{1}{2} - c + \frac{2c}{i_{\max} - K} \sum_{i\geq K} p_i (i - K), \end{aligned} \tag{22}$$

where we have used $\sum_i p_i = 1$ in the last equality. Eq. (22) shows that by setting $g_0 = \pi/4 - c$, we could recover $\mathbb{E}[\max(0, i-K)]$ from P_1 up to a scaling factor and a constant, from which we can subsequently recover the expected payoff $\mathbb{E}[f(i)]$ of the option given the probability distribution of the underlying asset. We should note that the fair value of the option requires appropriately discounting the expected payoff of the option to today, but as the discounting can be performed after the expectation value has been calculated, we omit it from our discussion for simplicity. We demonstrate the performance of our approach by running amplitude estimation using Qiskit [37] on the overall circuit produced by the elements described in this section, and verifying the convergence to the

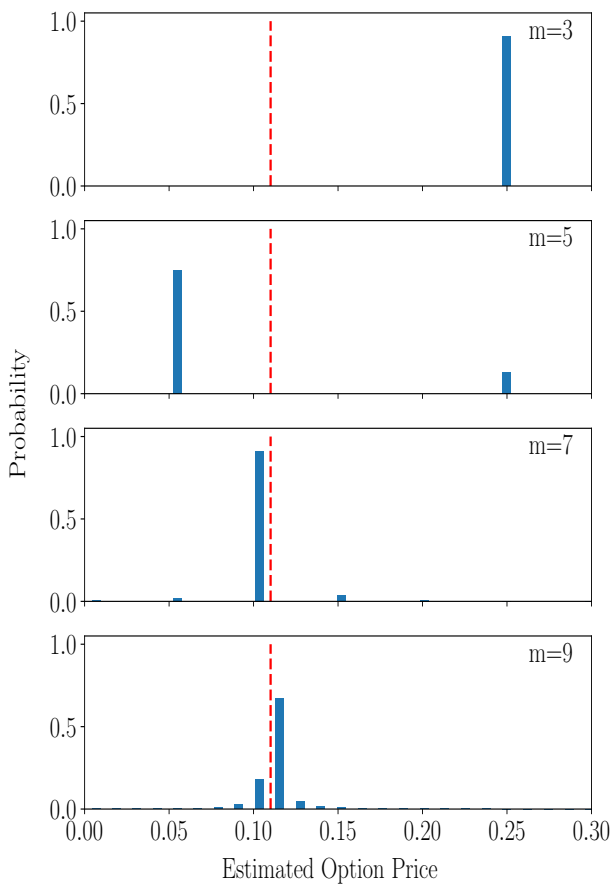

Figure 8: Results from applying amplitude estimation (Sec. 3.1) on a European call option with spot price distribution as given in Fig. 3 and a strike price $K = 2.0$, on a simulated quantum device with $m \in \{3, 5, 7, 9\}$ sampling qubits, i.e., $M \in \{8, 32, 128, 512\}$ quantum samples. The red dashed line corresponds to the (undiscounted) analytical value for this option, calculated using the Black-Scholes-Merton model. We limit the range of possible option values shown to $[0, 0.3]$ to illustrate the convergence of the estimation, as the cumulative probability in the windows shown exceeds 90% in each case.

analytically computed value or classical Monte Carlo estimate. An illustration of the convergence of a European call option with increasing evaluation qubits is shown in Fig. 8.

A straightforward extension of the analysis above yields a pricing model for a European put option, whose payoff $f(i) = \max(0, K - i)$ is equivalent to Eq. (17) with $a_> = b_> = 0$, $a_\leq = -1$, and $b_\leq = K$.

4.1.2 Portfolios of options

Various popular trading and hedging strategies rely on entering multiple option contracts at the same time instead of individual call or put options and as such, these strategies allow an investor to effectively construct a payoff that is more complex than that of vanilla options. For example, an investor who wants to profit from a volatile asset without picking a direction of where the volatility may drive the asset's price, may choose to enter a *straddle* option strategy, by buying both a call and a put option on the asset with the same expiration date and strike. If the underlying asset moves sharply up to expiration date, the investor can make a profit regardless of whether it moves higher or lower in value. Alternatively, the investor may opt for a *butterfly* option strategy by entering four appropriately structured option contracts with different strikes simultaneously. Because these option strategies give rise to piecewise linear payoff functions, the methodology described in the previous section can be extended to calculate the fair values of these option portfolios.

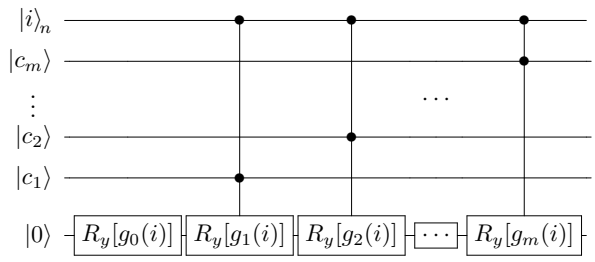

Figure 9: Quantum circuit that implements the multi-controlled Y-rotations for a portfolio of options with m strike prices.

In order to capture the structure of such option strategies, we can think of the individual options as defining one or more effective strike prices K_j and add a linear function $f_j(S) = a_j S + b_j$ between each of these strikes. For example, to price an option strategy with the payoff function

$$f_s(S) = \max(0, S - K_1) - \max(0, S - K_2), \quad (23)$$

which corresponds to a call spread (the option holder has purchased a call with strike K_1 and sold a call with strike K_2), we use functions f_0, f_1, and f_2 such that

$$f_s(S) = \begin{cases} f_0(S) & S < K_1, \\ f_0(S) + f_1(S) & K_1 \leq S < K_2, \\ f_0(S) + f_1(S) + f_2(S) & K_2 \leq S. \end{cases} \quad (24)$$

To match Eq. (23) with Eq. (24), we set $f_0(S) = 0$, $f_1(S) = S - K_1$ and $f_2(S) = -S + K_2$. In general, to price a portfolio of options with m effective-strike prices K_1, ..., K_m and $m+1$ functions $f_0(S)$, ..., $f_m(S)$, we need an ancilla qubit per strike to indicate if the underlying has reached the strike. This allows us to generalize the discussion from Sec. 4.1.1. We apply a multi-controlled Y-rotation with angle $g_j(i)$ if $i \geq K_j$ for each strike K_j with $j \in \{1, ..., m\}$. The rotation $g_0(i)$ is always applied, see the circuit in Fig. 9. The functions $g_j(i)$ are determined using the same procedure as in Sec. 4.1.1.

4.2 Multi-asset and path-dependent options

In this section we show how to price options with path-dependent payoffs as well as options on more than one underlying asset. In these cases, the payoff function depends on a multivariate distribution of random variables $\{S_j\}$ with $j \in \{1, ..., d\}$. The S_j's may represent one or several assets at discrete moments in time or a basket of assets at the option maturity. In both cases, the probability distribution of the random variables S_j are truncated to the interval $[S_{j,\min}, S_{j,\max}]$ and discretized using 2^{n_j} points so that they can be represented by d quantum registers where register j has n_j qubits. Thus, the multivariate distribution is represented by the probabilities $p_{i_1,...,i_d}$ that the underlying has taken the values i_1, ..., i_d with $i_j \in \{0, ..., 2^{n_j} - 1\}$. The quantum state that represents this probability distribution, a generalization of Eq. (14), is

$$|\psi\rangle_n = \sum_{i_1,...,i_d} \sqrt{p_{i_1,...,i_d}}\, |i_1\rangle_{n_1} \otimes ... \otimes |i_d\rangle_{n_d}\,, \tag{25}$$

with $n = \sum_j n_j$. Various types of options, such as Asian options or basket options, require us to compute the sum of the random variables S_j. The addition of the values in two quantum registers $|a, b\rangle \to |a, a + b\rangle$ may be calculated in quantum computers with adder circuits based on CNOT and Toffoli gates [38–40]. To this end we add an extra qubit register with n' qubits to serve as an accumulator. By recursively applying adder circuits we perform the transformation $|\psi\rangle_n |0\rangle_{n'} \to |\phi\rangle_{n+n'}$ where $|\phi\rangle_{n+n'}$ is given by

$$\sum_{i_1,...,i_d} \sqrt{p_{i_1,...,i_d}}\, |i_1\rangle_{n_1} \otimes ... \otimes |i_d\rangle_{n_d} \otimes |i_1 + ... + i_d\rangle_{n'}\,. \tag{26}$$

Here circuit optimization may allow us to perform this computation in-place to minimize the number of qubit registers needed. Now, we use the methods discussed in the previous section to encode the option payoffs into the quantum circuit.

4.2.1 Basket Options

A European style basket option is an extension of the single asset European option discussed in Sec. 4.1, only now the payoff depends on a weighted sum of d underlying assets. A call option on a basket has the payoff profile

$$f(S_{\text{basket}}) = \max(0, S_{\text{basket}} - K) \tag{27}$$

where $S_{\text{basket}} = \vec{w} \cdot \vec{S}$, for basket weights $\vec{w} = [w_1, w_2, \ldots, w_d]$, $w_i \in [0, 1]$, underlying asset prices at option maturity $\vec{S} = [S_1, S_2, \ldots S_d]$ and strike K. In the BSM model, the underlying asset prices are described by a multivariate log-normal distribution with probability density function [41]

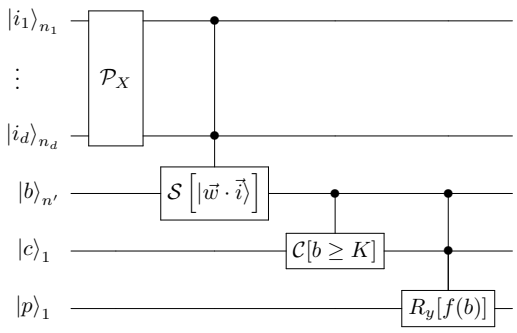

Figure 10: Schematic of the circuit that encodes the payoff of a basket call option of d underlying assets into the amplitude of a payoff qubit $|p\rangle$. First, a unitary $\mathcal{P}_X$ loads the multivariate distribution of Eq. (28) into d registers $|i_1\rangle_{n_1} \ldots |i_d\rangle_{n_d}$ using the methods described in Sec. 3.2. The weighted sum operator $\mathcal{S}$, see Appendix A, calculates the weighted sum $|w_1 \cdot i_1 + \ldots + w_d \cdot i_d\rangle$ into a register $|b\rangle_{n'}$ with n' qubits, where n' is large enough to hold the maximum possible sum. The comparator circuit $\mathcal{C}$ sets a comparator qubit $|c\rangle$ to $|1\rangle$ if $b \geq K$. Lastly, controlled-Y rotations are used to encode the option payoff $f(b) = \max(0, b - K)$ into the payoff qubit using the method shown in Fig. 7, controlled by the comparator qubit $|c\rangle$.

$$P(\vec{S}) = \frac{\exp\left(-\frac{1}{2}(\ln S - \mu)^T \Sigma^{-1} (\ln S - \mu)\right)}{(2\pi)^{d/2} (\det\Sigma)^{1/2} \prod_{i=1}^{d} S_i}, \tag{28}$$

where $\ln S = (\ln S_1, \ln S_2 \ldots, \ln S_d)^T$ and $\mu = (\mu_1, \mu_2 \ldots \mu_d)^T$, where each μ_i is the log-normal distribution parameter for each asset defined in the caption of Fig. 3. Σ is the $d \times d$ positive-definite covariance matrix of the d underlyings

$$\Sigma = T \begin{bmatrix} \sigma_1^2 & \rho_{12}\sigma_1\sigma_2 & \ldots & \rho_{1d}\sigma_1\sigma_d \\ \rho_{21}\sigma_2\sigma_1 & \sigma_2^2 & \ldots & \rho_{2d}\sigma_2\sigma_d \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{d1}\sigma_d\sigma_1 & \ldots & \ldots & \sigma_d^2 \end{bmatrix} \tag{29}$$

with σ_i the volatility of the ith asset, and $-1 \leq \rho_{ij} \leq 1$ the correlation between assets i and j and T the time to maturity.

The quantum circuit for pricing a European style basket call option is analogous to the single asset case, with an additional unitary to compute the weighted sum of the uncertainty registers $|i_1\rangle_{n_1} \ldots |i_d\rangle_{n_d}$ before applying the comparator and payoff circuits, controlled by the accumulator register $|b\rangle_{n'} = |i_1 + ... + i_d\rangle_{n'}$. A schematic of these components is shown in Fig. 10. The implementation details of the circuit that performs the weighted sum operator is discussed in Appendix A.

We use a basket option to compare the estimation accuracy between AE and classical Monte Carlo. From Eq. (2), we know that for M applications of the $\mathcal{Q}$ operator (see Fig. 1), the possible values returned by AE will be of the form $\sin^2(y\pi/M)$ for

$y \in \{0, ..., M-1\}$ and the maximum distance between two consecutive values is

$$\Delta_{\max} = \sin^2\left(\frac{\pi}{4} + \frac{2\pi}{4M}\right) - \sin^2\left(\frac{\pi}{4} - \frac{2\pi}{4M}\right). \quad (30)$$

This quantity determines how close M operations of $\mathcal{Q}$ can get us to the amplitude which encodes the option price. Using $\sin^2(\pi/4 + x) = x + 1/2 + \mathcal{O}(x^3)$ for $x \ll 1$, we get $\Delta_{\max} = \pi/M + \mathcal{O}(M^{-3})$ for $\pi/M \ll 1$. From Eq. (3) and Eq. (22), we can determine that with probability of at least $8/\pi^2$, our estimated option price using AE will be within

$$\Delta^O_{\max} = \frac{\pi/M}{2c} \times (i_{\max} - K) + \mathcal{O}(M^{-3}) \quad (31)$$

of the exact option price, where c, $i_{\max}$ and K are the parameters used to encode the option payoff into our quantum circuit, discussed in Sec. 4.1.1. To compare this estimation error to Monte Carlo, we use the same number of samples to price an option classically, and determine the approximation error at the same $8/\pi^2 \approx 81\%$ confidence interval by repeated simulations. The comparison for a basket option on three underlying assets shows that AE provides a quadratic speed-up, see Fig. 11. It is worth noting that for typical business cases, the number of paths required for acceptable pricing accuracy goes from tens of thousands to millions (depending on the option underlyings) [42, 43], so they are well within the range where amplitude estimation becomes more efficient than Monte Carlo, as shown in Fig. 11.

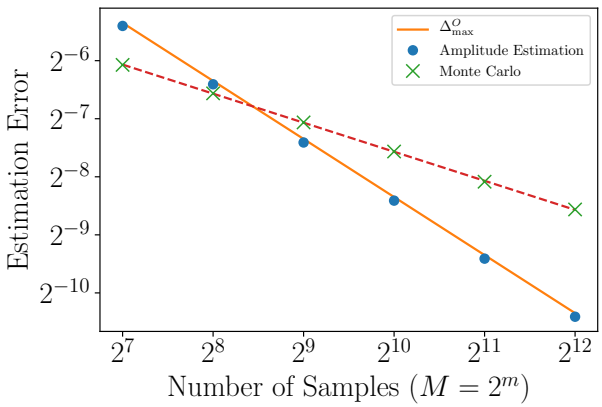

Figure 11: Comparison of the estimation error between Amplitude Estimation and Monte Carlo at the $8/\pi^2 \approx 81\%$ confidence interval for a basket option consisting of 3 identical, equally weighted assets with the parameters of Fig. 3, strike price $K = 2.0$ and asset correlations $\rho_{12} = \rho_{13} = \rho_{23} = 0.8$. The approximation error for amplitude estimation is plotted against the maximum expected error given by Eq. (31), illustrating the $\mathcal{O}(M^{-1})$ convergence. We calculate the equivalent Monte Carlo error at the same 81% confidence interval over 10,000 simulations for each sample number 2^m. The red dashed line shows a linear fit across the Monte Carlo errors, displaying the expected $\mathcal{O}(M^{-1/2})$ scaling.

4.2.2 Asian Options

We now examine arithmetic average Asian options which are single-asset, path-dependent options whose payoff depends on the price of the underlying asset at multiple time points before the option's expiration date. Specifically, the payoff of an Asian call option is given by

$$f(\bar{S}) = \max(0, \bar{S} - K) \quad (32)$$

where K is the strike price, $\bar{S}$ is the arithmetic average of the asset's value over a pre-defined number of points d between 0 and the option maturity T

$$\bar{S} = \frac{1}{d}\sum_{t=1}^{d} S_t. \quad (33)$$

The probability distribution of the asset price at time t will again be log-normal with probability density function

$$P(S_t) = \frac{1}{S_t\sigma\sqrt{2\pi\Delta t}} e^{-\frac{(\ln S_t - \mu_t)^2}{2\sigma^2\Delta t}}, \quad (34)$$

with $\mu_t = \left(r - 0.5\sigma^2\right)\Delta t + \ln(S_{t-1})$ and $\Delta t = T/d$. We can then use the multivariate distribution in Eq. (28), with $\vec{S}$ now a d-dimensional vector of asset prices at time points $[t_1 \dots t_d]$, instead of distinct underlying prices at maturity T. As we are not considering multiple underlying assets that could be correlated, the covariance matrix is diagonal $\Sigma = \Delta t[\mathrm{diag}(\sigma^2, ..., \sigma^2)]$. An illustration of the probability density function used for an asset defined on two time steps is shown in Fig. 12.

We now prepare the state $|\psi\rangle_n$, see Eq. (25), where each register represents the asset price at each time step up to maturity. Using the weighted sum operator of Appendix A with equal weights $1/d$, we then calculate the average value of the asset until maturity T, see Eq. (33), into a register $|\bar{S}\rangle$

$$|\underbrace{i_1}_{\Delta t}\underbrace{i_2}_{2\Delta t}\dots\underbrace{i_d}_{T}\rangle \mapsto |\bar{S}\rangle = |\frac{1}{d}\sum_{t=1}^{d} S_t\rangle. \quad (35)$$

Finally, we use the same comparator and rotation circuits that we employed for the basket option illustrated in Fig. 10 to load the payoff of an arithmetic average Asian option into the payoff qubit $|p\rangle$.

4.2.3 Barrier Options

Barrier options are another class of popular option types whose payoff is similar to vanilla European options, but they become activated or extinguished if the underlying asset crosses a pre-determined level called the *barrier*. In their simplest form, there are two general categories of barrier options

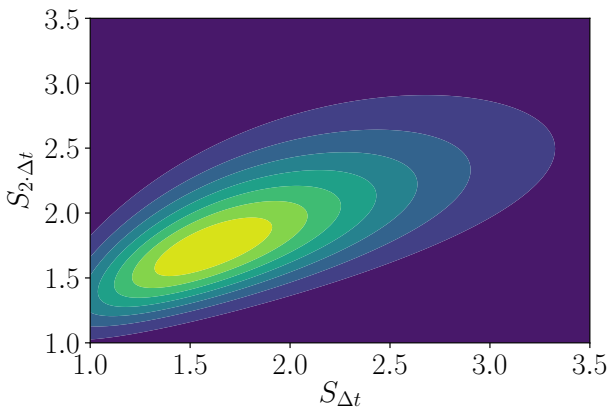

Figure 12: Probability density function of a multivariate log-normal distribution, see Eq. (28), for the asset shown in Fig. 3 defined on two time steps $t = \Delta t = T/2$ and $t = 2\Delta t = T$

- **Knock-Out** The option expires worthless if the underlying asset crosses a certain price level before the option's maturity.
- **Knock-In** The option has no value unless the underlying asset crosses a certain price level before maturity.

If the required barrier event for the option to have value at maturity occurs, the payoff then depends only on the value of the underlying asset at maturity and not on the path of the asset until then. If we consider a Knock-In barrier option and label the barrier level B, we can write the option's payoff as

$$f(S) = \begin{cases} \max(0, S_T - K) & \text{if } \exists t \text{ s.t. } S_t \geq B \\ 0 & \text{otherwise} \end{cases} \tag{36}$$

where T is the time to maturity, S_t the asset price at time t with $0 < t \leq T$ and K the option strike.

To construct a quantum circuit to price a Knock-In barrier option, we use the same method as for the Asian option where T is divided into d equidistant time intervals with $\Delta t = T/d$, and use registers $|i_1\rangle_{n_1} |i_2\rangle_{n_2} \ldots |i_d\rangle_{n_d}$ to represent the discretized range of asset prices at time $t \in \{\Delta t, 2\Delta t, \ldots, d \cdot \Delta t = T\}$. The probability distribution of Eq. (34) is used again to create the state $|\psi\rangle_n$ in Eq. (25).

To capture the path dependence introduced by the barrier, we use an additional d-qubit register $|b\rangle_d$ to monitor if the barrier is crossed. Each qubit $|b_t\rangle$ in $|b\rangle_d$ is set to $|1\rangle$ if $|i_t\rangle_{n_t} \geq B$. An ancilla qubit $|b_|\rangle$ is set to $|1\rangle$ if the barrier has been crossed in at least one time step. This is done by computing the logical OR, see Fig. 5, of every qubit in $|b\rangle_d$ and storing the result in the ancilla

$$|b_1 b_2 \ldots b_d\rangle |0\rangle \mapsto |b_1 b_2 \ldots b_d\rangle |b_1 \,||\, b_2 \ldots \,||\, b_d\rangle \,. \tag{37}$$

#	Single-qubit	CX	CCX	Depth
$m = 3$	2,091	2,056	90	3,927
$m = 5$	12,768	9,078	378	17,332
$m = 7$	52,275	37,132	1,530	70,916
$m = 9$	210,144	149,290	6,138	285,204

Table 2: Single-qubit, CNOT, Toffoli gate counts and overall circuit depth required for the full amplitude estimation circuits for each instance in Fig. 8, as a function of the number of sampling qubits m. These figures assume all-to-all connectivity across qubits.

This is computed with X (NOT) and Toffoli gates and $d - 2$ ancilla qubits. The ancilla qubit $|b_|\rangle$ is then used as a control for the payoff rotation into the payoff qubit, effectively knocking the option *in*. For Knock-Out barrier options, we can follow the same steps and apply an X gate to the ancilla barrier qubit before using it as control, in this manner knocking the option *out* if the barrier level has been crossed. A circuit displaying all the components required to price a Knock-In barrier option is shown in Fig. 13. Results from amplitude estimation on a barrier option circuit using a quantum simulator are shown in Fig. 14.

Even though we have focused our attention on barrier options where the barrier event is the underlying asset crossing a barrier from below, we can use the same method to price barrier options where barrier events are defined as the asset crossing the value from above. This only requires changing the comparator circuits to compute $S_t \leq B$ in the barrier register $|b\rangle_d$.

For all path-dependent options, including the Asian and barrier options we have examined in this section, we note that the choice of time intervals on which we need to represent the probability distribution on quantum registers depends on the type of option in question and the type of underlying(s). For instance, when pricing barrier options, we only need to represent the probability distribution at the barrier dates and at maturity. However, as the choice of which time intervals need to be represented for option pricing is independent of whether we employ a quantum or a classical pricing model, a detailed analysis of this choice is beyond the scope of this work.

5 Quantum hardware results

In this section we examine the performance of European call option circuits evaluated on quantum hardware. Quantum circuits based on standard amplitude estimation are not promising candidates for near-term devices, given that they require extra qubits to control the accuracy of the calculation, and multi-controlled gate operations. Tab. 2 lists the number of single and multi-qubit gates required for the European call op-

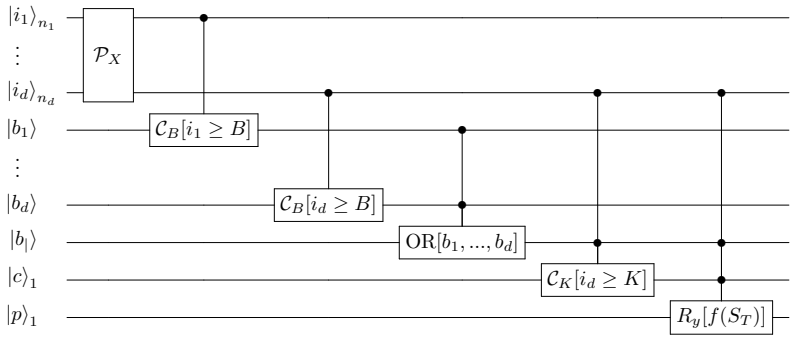

Figure 13: Circuit that encodes the payoff of a Knock-In barrier option in the state of an ancilla qubit $|p\rangle_1$. The unitary operator $\mathcal{P}_X$ is used to initialize the state of Eq. (25). Comparator circuits $\mathcal{C}_B$ are used to set a barrier qubit b_j for all $j \in [1, d]$ if the asset price represented by $|i_j\rangle$ crosses the barrier B. The logical OR of all b_j qubits is computed into ancilla $|b_|\rangle$. The strike comparator circuit $\mathcal{C}_K$ sets the comparator qubit $|c\rangle_1$ to $|1\rangle$ if the asset price at maturity $|i_d\rangle \geq K$. Finally, Y-rotations encode the payoff qubit $|p\rangle_1$, controlled on $|i_d\rangle$, the strike qubit $|c\rangle_1$ and the barrier qubit $|b_|\rangle$ which is $|1\rangle$ only if the asset price has crossed the barrier at least once before maturity.

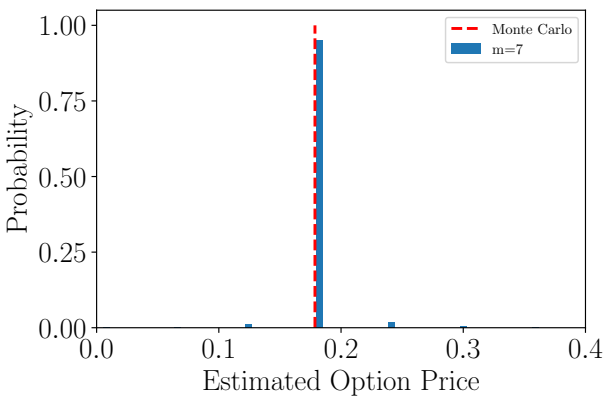

Figure 14: Estimated option price for a barrier option using amplitude estimation on a quantum simulator. The option is defined on the asset of Fig. 3 with two timesteps $T/2$ and $T = 300/365$ and 2 qubits used to represent the uncertainty per timestep. The option strike is $K = 1.9$ and a barrier was added at $B = 2.0$ on both timesteps. The red dotted line is the (undiscounted) value of the option calculated with classical Monte Carlo and $100,000$ paths and the blue bars show the estimated option values using amplitude estimation with $m = 7$ sampling qubits.

tion examples in Fig. 8, all of which are far from the reach of current and near-term quantum hardware.

However, a quadratic speed-up is also possible by performing AE without quantum phase estimation (see Sec. 3.1). Even though removing phase estimation from amplitude estimation does not make the overall algorithm immediately compatible with noisy quantum computers, it does lead to significantly shorter circuits than the original implementation of AE, allowing us to examine how it performs on noisy quantum hardware.

We hence focus on the circuits required to perform AE without phase estimation and show results for a European call option, using three qubits, two of which represent the uncertainty and one encodes the payoff. We consider a log-normal random distribution with $S_0 = 2$, $\sigma = 40\%$, $r = 5\%$, and $T = 40/365$, see Fig. 3, and truncate the distribution to the interval defined by three standard deviations around the mean. With two qubits encoding this distribution, the possible values are $[1.21, 1.74, 2.28, 2.81]$, represented by $|00\rangle, \ldots, |11\rangle$, with corresponding probabilities $0.1\%, 55.4\%, 42.5\%$, and 1.9%. We set the strike price to $K = 1.74$.

To examine the behavior of the circuit for different input probability distributions, we run eight experiments that differ by the initial spot price S_0 and all other parameters are kept constant. The spot price is varied from 1.8 to 2.5 in increments of 0.1. This way we can use the same circuit for all experiments and only vary the Y-rotation angles used to map the initial probability distribution onto the qubit register. This choice of input parameters allows us to evaluate our circuits for expected option prices in the range $[0.0754, 0.7338]$.

Following the procedure detailed in Sec. 3.1, we construct the circuits for $\mathcal{A}|0\rangle_3$ and $\mathcal{QA}|0\rangle_3$, which correspond to $k = 0, 1$ (i.e. $m = 1$) in Eq. (4), with $n = 2$.

We then perform repeated measurements of the circuits, and by combining the measured probabilities for all k in a single likelihood function, we can perform a maximum likelihood estimation for θ_a, hence obtaining an estimate for $a = \mathbb{E}[f(S)]$ (see Eq. (6)), i.e. the expected payoff. Each experiment is evaluated on the *IBM Q Tokyo* 20-qubit device with 8192 shots. We repeat each 8192-shot experiment 20 times and average over the 20 measured probabilities in order to limit the effect of any one-off issues with the device. The standard deviation of the measured probabilities across the 20 runs was always $< 2\%$. The connectivity of *IBM Q Tokyo* allows to choose three fully connected qubits for the experiments, and thus, no swaps are required to implement any two-qubit gate in our circuits [37]. For all circuits described in the following sections, we used qubits 6, 10 and 11.

Note that even though we are only interested in the result of a single qubit, we always measure all three qubits to be able to apply readout error mitigation as discussed later in Sec. 5.2.

5.1 Algorithm and Operators

We now show how to construct the operator $\mathcal{A}$ that is required for AE. The log-normal distribution on two qubits can be loaded using a single CNOT gate and four single-qubit rotations [44]. To encode the payoff function, we also exploit the small number of qubits and apply a uniformly controlled Y-rotation instead of the generic construction using comparators introduced in Sec. 4. A uniformly controlled Y-rotation, i.e.

$$|i\rangle_n |0\rangle \rightarrow |i\rangle_n R_y(\theta_i) |0\rangle , \tag{38}$$

implements a different rotation angle θ_i, $i = 0, ..., 2^n - 1$ for each state of the n-control qubits. For $n = 2$, this operation can be efficiently implemented using four CNOT gates and four single qubit Y-rotations [45, 46]. The resulting circuit implementing $\mathcal{A}$ is shown in Fig. 15. Note that in our setup, different initial distributions only lead to different angles of the first four Y-rotations and do not affect the rest of the circuit. Although we use a uniformly controlled rotation, the rotation angles are constructed in the same way described in Sec. 3.3. We use an approximation scaling of $c = 0.25$ and the resulting angles are $[\theta_0, \ldots, \theta_3] = [1.1781, 1.1781, 1.5708, 1.9635]$, which shows the piecewise linear structure of the payoff function.

The total resulting circuit requires five CNOT gates and eight single-qubit Y-rotations, see Fig. 15. Since we use uniformly controlled rotations, we do not need any ancilla qubit. Note that if we want to evaluate the circuit for $\mathcal{A}$ alone, we can replace the last CNOT gate in Fig. 15 by classical post-processing of the measurement result: if q_1 is measured as $|1\rangle$, we flip q_2 and otherwise we do nothing. This further reduces the overall CNOT gate count to four.

In the remainder of this section, we focus on $\mathcal{QA}|0\rangle_3$, i.e., the outlined algorithm for $m = 1$, to examine the reach of today's quantum hardware in evaluating AE option pricing circuits which do not require phase estimation. We note that this evaluation is relevant to any quantum algorithm realizing AE without phase estimation and is independent of the approach described in [22] or any other particular implementation.

After optimzing the gate count, the resulting circuit for $\mathcal{QA}|0\rangle_3$ consists of 18 CNOT gates and 33 single-qubit gates. The detailed circuit diagram and applied circuit optimization steps are provided in Appendix B.

5.2 Error mitigation and results

We run the circuits for $\mathcal{A}|0\rangle_3$ and $\mathcal{QA}|0\rangle_3$ on noisy quantum hardware. The results are affected by readout errors and errors that occur during the execution of the circuits.

To mitigate readout errors we run a calibration sequence in which we individually prepare and measure all eight basis states [37, 47]. The result is a 8×8 readout-matrix $\mathcal{R}$ that holds the probability of measuring each basis state as function of the basis state in which the system was prepared. We use $\mathcal{R}$ to correct all subsequent measurements. The error we measure on P_1 for $\mathcal{A}|0\rangle_3$ was reduced from $\sim 6\%$ to $\sim 4\%$ using readout error mitigation.

Errors occuring during the quantum circuit can be mitigated using Richardson extrapolation [48]. First, the quantum circuit is run using a rescaled Hamiltonian to amplify the effect of the noise. Second, a Richardson extrapolation is used to extract the result of the quantum circuit at the zero noise limit. In hardware, error mitigation is done by stretching the duration of the gates. For each stretch factor the qubit gates need to be recalibrated [8]. Here, we use a simplified error mitigation protocol that circumvents the need to recalibrate the gates but still allows us to increase the accuracy of the quantum hardware [49]. Since the single-qubit and CNOT gates have an average randomized benchmarking fidelity of 99.7% and 97.8%, respectively, we restrict our error mitigation to the CNOT gates. Furthermore, because the optimized circuit for $\mathcal{A}|0\rangle_3$ contains only 4 CNOT gates, we employ the error mitigation protocol only when evaluating $\mathcal{QA}|0\rangle_3$ which consists of 18 CNOT gates.

We run the circuit for $\mathcal{QA}|0\rangle_3$ three times. In each run we replace the CNOT gates of the original circuit by one, three and five CNOT gates for a total of 18, 54, and 90 CNOT gates, respectively. Since a pair of perfect CNOT gates simplifies to the identity these extra gates allow us to amplify the error of the CNOT gate without having to stretch the gate duration, thus, avoiding the need to recalibrate the gate parameters. As the number of CNOT gates is

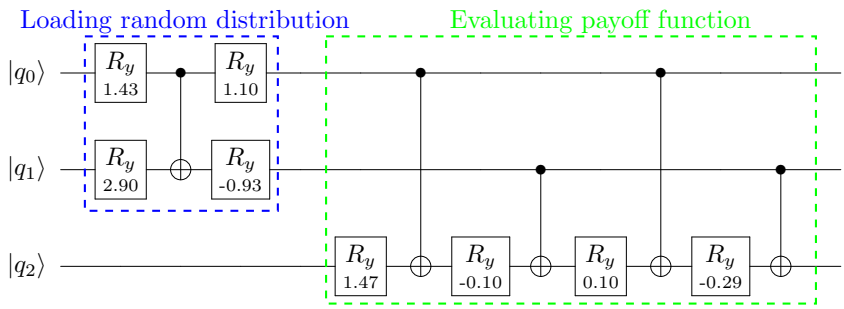

Figure 15: The $\mathcal{A}$ operator of the considered European call option: first, the 2-qubit approximation of a log-normal distribution is loaded, and second, the piecewise linear payoff function is applied to last qubit controlled by the first two. This operator can be used within amplitude estimation to evaluate the expected payoff of the corresponding option.

increased the probability of measuring $|1\rangle$ tends towards 0.5 for all initial spot prices, see Fig. 16(b). After applying a second-order Richardson extrapolation, i.e quadratic extrapolation, we recover the same behavior as the option price obtained from classical simulations, see Fig. 16(c). Our simple error mitigation scheme dramatically increased the accuracy of the calculated option price: it reduced the error, averaged over the initial spot price, from 62% to 21%.

6 Conclusion

We have presented a methodology and the quantum circuits to price options and option portfolios on a gate-based quantum computer. We showed how to account for some of the more complex features present in exotic options such as path-dependency with barriers and averages. The results that we show are available in the finance module in *Qiskit* [37]. Future work may involve calculating the price derivatives [50] with a quantum computer. Pricing options relies on AE. This quantum algorithm allows a quadratic speed-up compared to traditional Monte Carlo simulations, but will most likely require a universal fault-tolerant quantum computer [51]. However, research to improve the algorithms is ongoing [52–54]. Here we have used a new algorithm [22] that retains the AE speed-up but that uses less gates to measure the price of an option. Furthermore, we employed a simple error mitigation scheme that allowed us to greatly reduce the errors from the noisy quantum hardware. However, larger quantum hardware capable of running deeper quantum circuits with more qubits than the currently available quantum computers is needed to price the typical portfolios seen in the financial industry. Future work could focus on reducing the number of quantum registers in our implementation by performing some of the computation in-place.

7 Acknowledgments

The authors want to thank Abhinav Kandala for the very constructive discussions on error mitigation and real quantum hardware experiments. C.Z. and R.I. acknowledge the support of the National Centre of Competence in *Research Quantum Science and Technology* (QSIT).

Opinions and estimates constitute our judgment as of the date of this Material, are for informational purposes only and are subject to change without notice. This Material is not the product of J.P. Morgan's Research Department and therefore, has not been prepared in accordance with legal requirements to promote the independence of research, including but not limited to, the prohibition on the dealing ahead of the dissemination of investment research. This Material is not intended as research, a recommendation, advice, offer or solicitation for the purchase or sale of any financial product or service, or to be used in any way for evaluating the merits of participating in any transaction. It is not a research report and is not intended as such. Past performance is not indicative of future results. Please consult your own advisors regarding legal, tax, accounting or any other aspects including suitability implications for your particular circumstances. J.P. Morgan disclaims any responsibility or liability whatsoever for the quality, accuracy or completeness of the information herein, and for any reliance on, or use of this material in any way. Important disclosures at: www.jpmorgan.com/disclosures

IBM, IBM Q, Qiskit are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product or service names may be trademarks or service marks of IBM or other companies.

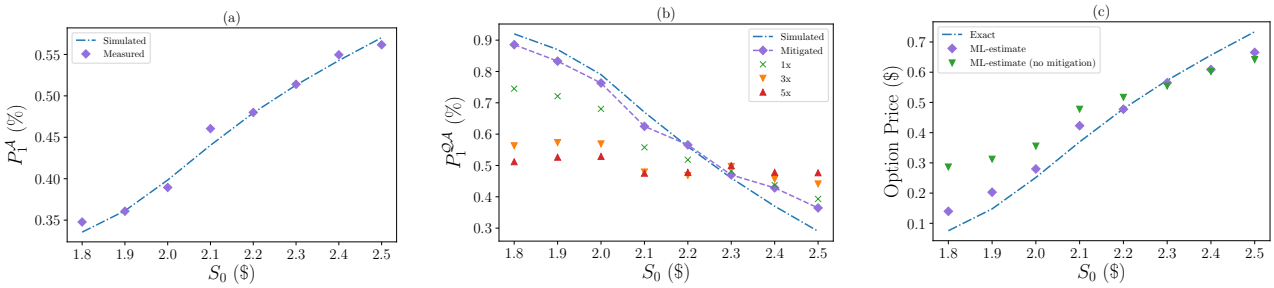

Figure 16: Error-mitigated hardware results for $\mathcal{A}\left|0\right\rangle_3$, $\mathcal{QA}\left|0\right\rangle_3$ and the estimated option price after applying maximum likelihood estimation as a function of the initial spot price S_0. (a) Probability of measuring $\left|1\right\rangle$ for the $\mathcal{QA}\left|0\right\rangle_3$ circuit (see Appendix B, Fig. 15) (b) Probability of measuring $\left|1\right\rangle$ for the $\mathcal{QA}\left|0\right\rangle_3$ circuit (see Fig. 19). We show the measured probabilities when replacing each CNOT by one, three and five CNOT gates (green, orange, red, respectively), the zero-noise limit calculated using a second-order Richardson extrapolation method (purple), and the probability measured using the simulator (blue). (c) Option price estimated with maximum likelihood estimation from measurements of $\mathcal{QA}\left|0\right\rangle_3$ and $\mathcal{A}\left|0\right\rangle_3$ with error mitigation (purple) and without (green). The exact option price for each initial spot price S_0 is shown in blue.

A Circuit implementation of weighted sum operator

A.1 Weighted sum of single qubits

In this appendix, we demonstrate an implementation of the weighted sum operator on a quantum circuit. The weighted sum operator $\mathcal{S}$ computes the arithmetic sum of the values of n qubits $\left|a\right\rangle_n = \left|a_1 \dots a_n\right\rangle$ weighted by n classically defined non-negative integer weights $\omega = (\omega_1, \omega_2, \dots, \omega_n)$, and stores the result into another m-qubit register $\left|s\right\rangle_m = \left|s_1 \cdots s_m\right\rangle$ initialized to $\left|0\right\rangle_m$. In other words,

$$\mathcal{S}\left|a\right\rangle_n \left|0\right\rangle_m = \left|a\right\rangle_n \left|\sum_{i=1}^{n} \omega_i a_i\right\rangle_m , \tag{39}$$

where

$$m = \left\lfloor \log_2 \left(\sum_{i=0}^{n} \omega_i \right) \right\rfloor + 1. \tag{40}$$

The choice of m ensures that the sum register $\left|s\right\rangle_m$ is large enough to hold the largest possible weighted sum, i.e. the sum of all weights. Alternatively, we can write the weights in the form of a binary matrix $\Omega = (\Omega_{i,j}) \in \{0,1\}^{n \times n^*}$, where the i-th row in Ω is the binary representation of weight ω_i and $n^* = \max_{i=1}^{d} n_i$. We use the convention that less significant digits have smaller indices, so $\left|s_1\right\rangle$ and $\Omega_{i,1}$ are the least significant digits of the respective binary numbers. Using this binary matrix representation, $\mathcal{S}$ is to add the i-th qubit $\left|a_i\right\rangle$ of the state register to the j-th qubit $\left|s_j\right\rangle$ of the sum register if and only if $\Omega_{i,j} = 1$. Depending on the values of the weights, an additional quantum register may be necessary to temporarily store the carries during addition operations. We use $\left|c_j\right\rangle$ to denote the ancilla qubit used to store the carry from adding a digit to $\left|s_j\right\rangle$. These ancilla qubits are initialized to $\left|0\right\rangle$ and will be reset to their initial states at the end of the computation.

Based on the above setup, we build quantum circuits for the weighted sum operator using three elementary gates: X (NOT), CNOT, and the Toffoli gate (CCNOT). These three gates suffice to build any Boolean function [38]. Starting from the first column in Ω, for each column j, we find all elements with $\Omega_{i,j} = 1$ and add the corresponding state qubit $\left|a_i\right\rangle$ to $\left|s_j\right\rangle$. The addition of two qubits involves three operations detailed in Fig. 17: (a) computation of the carry using a Toffoli gate ($\mathcal{M}$), (b) computation of the current digit using a CNOT ($\mathcal{D}$), (c) reset of the carry computation using two X gates and one Toffoli gate ($\widetilde{\mathcal{M}}$). When adding $\left|a_i\right\rangle$ to the j-th qubit of the sum register, the computation starts by applying $\mathcal{M}$ and then $\mathcal{D}$ to $\left|a_i\right\rangle$, $\left|s_j\right\rangle$ and $\left|c_j\right\rangle$, which adds $\left|a_i\right\rangle$ to $\left|s_j\right\rangle$ and stores the carry into $\left|c_j\right\rangle$. Then, using the same two operations, it adds the carry $\left|c_j\right\rangle$ to the next sum qubit $\left|s_{j+1}\right\rangle$ with carry recorded in $\left|c_{j+1}\right\rangle$. The process is iterated until all carries are handled. Finally, it resets the carry qubits by applying $\widetilde{\mathcal{M}}$ in reverse order of the carry computation. We reset the carry qubits in order to reuse them in later computations if necessary.

In general, we need $\max(k-2, 0)$ carry qubits to compute the addition of $\left|a_i\right\rangle$ on $\left|s_j\right\rangle$, where $k \geq 1$ is the smallest integer satisfying

$${}_k\left\langle 1\right|\rho^s_{j,j+k-1}\left|1\right\rangle_k = 0, \tag{41}$$

where $\rho^s_{j,j+k-1}$ is the density matrix corresponding to $\left|s_j \cdots s_{j+k-1}\right\rangle$. In the $k = 1$ case, i.e. $\left|s_j\right\rangle = 0$, the computation is reduced to "copying" $\left|a_i\right\rangle$ to $\left|s_j\right\rangle$ using the bit addition operator $\mathcal{D}$, and no carries would be produced. For $k \geq 2$, Eq. (41) guarantees no carries from $\left|s_{j+k-1}\right\rangle$ and beyond. Therefore we can directly compute the carry from $\left|s_{j+k-2}\right\rangle$ into $\left|s_{j+k-1}\right\rangle$ without worrying about additional carries. This eliminates the need for an ancilla qubit $\left|c_{j+k-2}\right\rangle$, and hence the number of carry qubits needed is $k-2$. To further reduce the number of ancilla qubits, we can use any sum qubit $\left|s_j\right\rangle = \left|0\right\rangle$ during the computation. In our

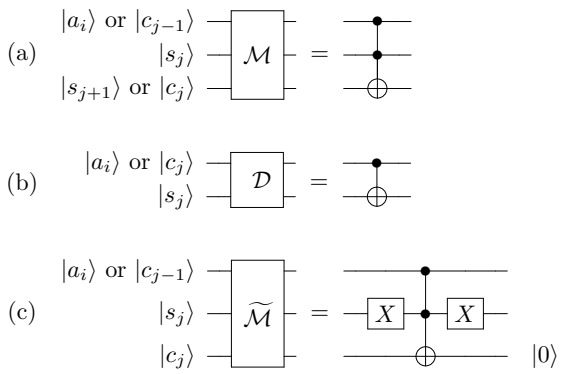

Figure 17: Three component gates used to construct the weighted sum operator $\mathcal{S}$. (a) The carry operator $\mathcal{M}$ consisting of one Toffoli gate, which computes the carry from adding $|a_i\rangle$ (or $|c_{j-1}\rangle$) and $|s_j\rangle$ into $|s_{j+1}\rangle$ or $|c_j\rangle$. (b) The bit addition operator $\mathcal{D}$ consisting of one CNOT gate, which adds the state qubit $|a_i\rangle$ or the carry qubit from the previous digit $|c_{j-1}\rangle$ to the sum qubit $|s_j\rangle$. (c) The carry reset operator $\widetilde{\mathcal{M}}$ consisting of two X gates and one Toffoli gate, which resets the carry qubit $|c_j\rangle$ back to $|0\rangle$.

case, since we are processing Ω column by column, all sum qubits more significant than $|s_{j+k-1}\rangle$ would be $|0\rangle$. In other words, we have the last $m-(j+k-1)$ sum qubits usable as carry qubits in the computation described above.

As the weights are known at the time of building the circuit, the possible states that $|s\rangle_m$ can have before each addition of a state qubit $|a_i\rangle$ are also computable. Since we are adding $|a_i\rangle$ to $|s\rangle_m$ starting from the least significant bit, k equals the bit length of the maximum possible sum on $|s_j \dots s_m\rangle$ after adding $|a_i\rangle$ to $|s_j\rangle$. In other words,

$$k = \log_2 \left\lfloor \sum_{\substack{u\le i,\text{ or}\\ v\le j}} \frac{\Omega_{u,v}}{2^{j-v}} \right\rfloor + 1. \tag{42}$$

Therefore, the number of carry operations and additional ancilla qubits required for each addition of $|a_i\rangle$ can be determined. The term in the $\lfloor\cdot\rfloor$ in Eq. (42) is upper-bounded by

$$\sum_{\substack{u\le i,\text{ or}\\ v\le j}} \frac{\Omega_{u,v}}{2^v} \le \sum_{j=1}^{m} \frac{n_{\max}}{2^{j-1}} < 2n_{\max} \le 2n, \tag{43}$$

where $n_{\max} = \max_{j=1}^{m} \sum_{i=1} n\Omega_{i,j}$ is the maximum number of 1's in a column of Ω. It immediately follows that the number of non-trivial carry operations (i.e. carry operations that requires $\widetilde{\mathcal{M}}$) required to add $|a_i\rangle$ to $|s_j \dots s_m\rangle$ is upper-bounded by

$$k-2 < \log_2 \lfloor n_{\max} \rfloor \le \log_2 \lfloor n \rfloor, \tag{44}$$

and the number of ancilla qubits required for the entire implementation of $\mathcal{S}$ is at most the upper bound for $k-2$, since we may use some sum qubits as carries. In other words, the number of ancilla qubits required for $\mathcal{S}$ grows at most logarithmically with the number of state qubits n.

A.2 Sum of multi-qubit integers

The weighted sum operator $\mathcal{S}$ can be used to calculate the sum of d multi-qubit positive integers on a quantum register. To do that we first prepare the input register in the state

$$|a\rangle_n = |a_1^{(1)} \dots a_{n_1}^{(1)} \dots a_1^{(d)} \dots a_{n_d}^{(d)}\rangle, \quad n = \sum_{i=1}^{d} n_i, \tag{45}$$

where $|a_1^{(i)} \dots a_{n_i}^{(i)}\rangle, i \in [1,d]$ is the binary representation of the i-th integer to sum with n_i qubits, least significant figure first. Then we set the weights as

$$\omega = (2^0, \dots, 2^{n_1-1}, \dots, 2^0, \dots, 2^{n_d-1}), \tag{46}$$

or equivalently,

$$\Omega_{n\times n^*} = \left(I^T_{n_1\times n^*}, \dots, I^T_{n_d\times n^*}\right)^T, \tag{47}$$

where $I_{n_i\times n^*} = \left(I_{n_i}, 0_{n_i\times(n^*-n_i)}\right), i \in [1,d]$ and I_{n_i} is the n_i-dimensional identity matrix. Now if we build a weighted sum operator based on the weights in Eq. (46) and apply it on the input state qubits in Eq. (45), we would have the sum of the d integers in $|s\rangle_m$.

Fig. 18 shows an example circuit computing the sum of two 3-digit binary numbers represented on a 6-qubit quantum register $|a\rangle_3 |b\rangle_3$, and storing the result into a 4-qubit register $|s\rangle_4$. The circuit is implemented by a weighted sum operator $\mathcal{S}$ with weights $\omega = (1,2,4,1,2,4)$. The addition of each qubit onto the sum qubits requires one carry gate ($\mathcal{M}$) followed by one addition gate ($\mathcal{D}$), except for the first bit $|a_1\rangle$ which does not have any carries before its addition. This results in a total of 6 CNOT ($\mathcal{D}$) gates and 5 Toffoli ($\mathcal{M}$) gates. The 11 gates are grouped in three groups, as is shown in Fig. 18 by dashed boxes. Each group computes the sum of the bits $|a_j\rangle$ and $|b_j\rangle$ into $|s_j\rangle$ and the carry into $|s_{j+1}\rangle$. Note that separate carry qubits are not required, therefore no carry reset operators $\widetilde{\mathcal{M}}$ are used. In fact, using the above construction for $\mathcal{S}$, no extra carry qubits will be required for the addition of any two binary numbers. In general, $\mathcal{S}$ requires at most $\lfloor \log_2 d \rfloor$ ancilla qubits for carrying operations, which directly comes from Eq. (44).

A.3 Weighted sum of multi-qubit integers

In addition to summing up d integers equally, a weight w_i may also be added to each integer $a^{(i)}$. In that case, the weight matrix would be

$$\Omega = \left(w_1 \cdot I^T_{n_1\times n^*}, \dots, w_d \cdot I^T_{n_d\times n^*}\right)^T. \tag{48}$$

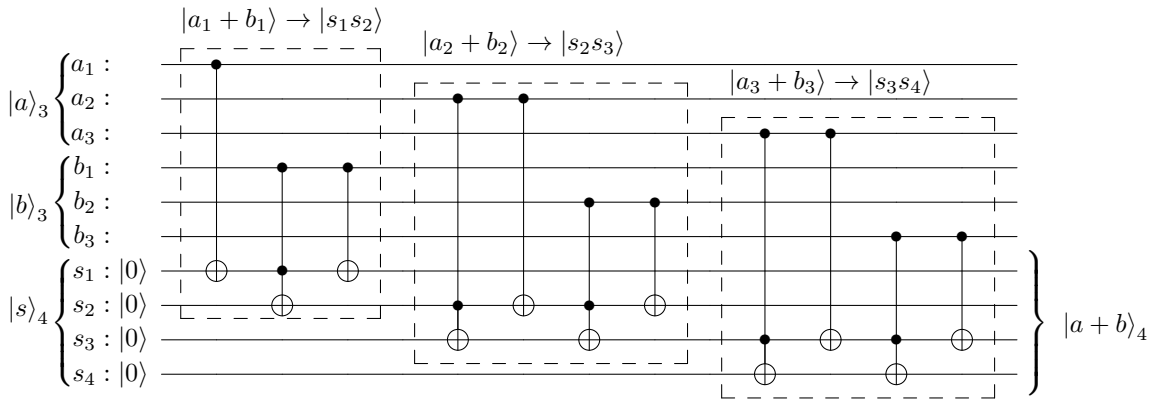

Figure 18: A circuit computing the sum of binary numbers $|a\rangle_3$ and $|b\rangle_3$ into $|s\rangle_4$ implemented using the weighted sum operator with weights $\omega = (1, 2, 4, 1, 2, 4)$.

In the case where w_i are not integers, we can rescale the values represented on the quantum register by a common factor to make all weights integers. For example, if we are adding two numbers with weights 0.2 and 0.8, we could use integer weights of $w_1 = 1$ and $w_2 = 4$ instead, and reinterpret the resulting sum in postprocessing by dividing it by 5.

B Optimized Circuit for $\mathcal{QA}\,|0\rangle_3$

In the following, we describe the circuit used for $\mathcal{QA}\,|0\rangle_3$ requiring only 18 CNOT gates. We have that $\mathcal{Q} = -\mathcal{A}\mathcal{S}_0\mathcal{A}^\dagger\mathcal{S}_{\psi_0}$, where $\mathcal{S}_0 = 1 - 2\,|0\rangle\langle 0|$ and $\mathcal{S}_{\psi_0} = 1 - 2\,|\psi_0\rangle\,|0\rangle\,\langle\psi_0|\,\langle 0|$ perform reflections on $|0\rangle_3$ and $|\psi_0\rangle_2\,|0\rangle$, respectively. S_{ψ_0} can be implemented up to a global phase using a single-qubit Z-gate on the last qubit, which is sufficient to differentiate between $|\psi_0\rangle\,|0\rangle$ and $|\psi_1\rangle\,|1\rangle$. $\mathcal{S}_0$ is a bit more difficult and we use circuit synthesis for diagonal unitary matrices to achieve an efficient decomposition into gates [55]. This construction lead to 16 CNOT gates for $\mathcal{Q}$ and 21 for $\mathcal{QA}$, which was still a bit too much to run on real hardware.

To further reduce the CNOT count, we look at the full circuit $\mathcal{QA}\,|0\rangle_3$ and we applied the following optimization steps. The last part in $\mathcal{Q}$ is the application of $\mathcal{A}$. As mentioned in Sec. 5, we can drop the very last CNOT gate and apply it in a classical postprocessing. Furthermore, in $\mathcal{QA}\,|0\rangle_3$, we have S_{ψ_0} between $\mathcal{A}$ and $\mathcal{A}^\dagger$, i.e. $\mathcal{A}^\dagger S_{\psi_0}\mathcal{A}$, where the uniformly controlled Y-rotations in $\mathcal{A}$ ($\mathcal{A}^\dagger$) are right before (after) S_{ψ_0}. On the other hand, the Z-gate that implements S_{ψ_0} can be decomposed into an X-rotation and a Y-rotation. The Y-rotation can subsequently be absorbed into one of the uniformly controlled Y-rotations in $\mathcal{A}$ or $\mathcal{A}^\dagger$, changing the angles accordingly. Since the remaining X-rotation commutes with the two neighboring CNOT gates from $\mathcal{A}$ and $\mathcal{A}^\dagger$, we can move the X-rotation so that the two CNOT gates cancel each other. This reduces the CNOT gate count for $\mathcal{QA}\,|0\rangle_3$ to 18 and the resulting circuit is reported in Fig. 19.

References

[1] John C. Hull, *Options, futures, and other derivatives*, 6th ed. (Pearson Prentice Hall, Upper Saddle River, NJ [u.a.], 2006).

[2] Fischer Black and Myron Scholes, "The pricing of options and corporate liabilities," Journal of Political Economy **81**, 637–654 (1973).

[3] Bruno Dupire, "Pricing with a smile," Risk Magazine , 18–20 (1994).

[4] Phelim P. Boyle, "Options: A Monte Carlo approach," Journal of Financial Economics **4**, 323–338 (1977).

[5] Paul Glasserman, *Monte Carlo Methods in Financial Engineering* (Springer-Verlag New York, 2003) p. 596.

[6] Michael A. Nielsen and Isaac L. Chuang, *Cambridge University Press* (2010) p. 702.

[7] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M. Chow, and Jay M. Gambetta, "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," Nature **549**, 242 (2017).

[8] Abhinav Kandala, Kristan Temme, Antonio D. Corcoles, Antonio Mezzacapo, Jerry M. Chow, and Jay M. Gambetta, "Error mitigation extends the computational reach of a noisy quantum processor," Nature **567**, 491–495 (2019).

[9] N. Moll, P. Barkoutsos, L. S. Bishop, J. M. Chow, A. Cross, D. J. Egger, S. Filipp, A. Fuhrer, J. M. Gambetta, M. Ganzhorn, A. Kandala, A. Mezzacapo, P. Müller, W. Riess, G. Salis, J. Smolin, I. Tavernelli, and K. Temme, "Quantum optimization using variational algorithms on near-

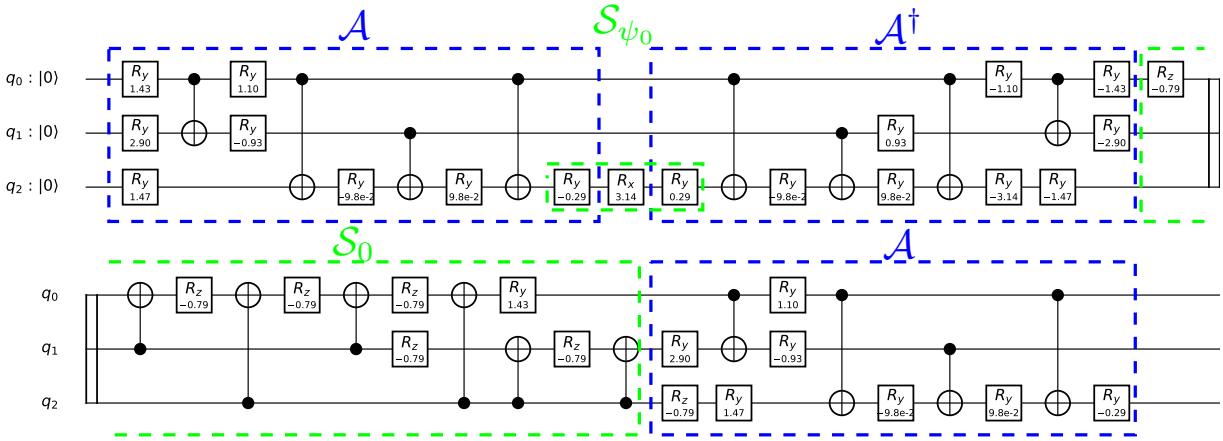

Figure 19: The optimized circuit for $\mathcal{QA}\,|0\rangle_3$ used for the experiments on real quantum hardware. It requires 18 CNOT gates and 33 single qubit gates. The initial spot price is assumed to be equal to 2. The dashed boxes indicate which parts are used for $\mathcal{A}$, $\mathcal{A}^\dagger$, $\mathcal{S}_{\psi_0}$, and $\mathcal{S}_0$. Note that due to the circuit optimization, some boxes are slightly overlapping.

term quantum devices," Quantum Science and Technology **3**, 030503 (2018).

[10] M. Ganzhorn, D.J. Egger, P. Barkoutsos, P. Ollitrault, G. Salis, N. Moll, M. Roth, A. Fuhrer, P. Mueller, S. Woerner, I. Tavernelli, and S. Filipp, "Gate-efficient simulation of molecular eigenstates on a quantum computer," Phys. Rev. Applied **11**, 044092 (2019).

[11] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd, "Quantum algorithm for linear systems of equations," Phys. Rev. Lett. **103**, 150502 (2009).

[12] Seth Lloyd, Masoud Mohseni, and Patrick Rebentrost, "Quantum principal component analysis," Nature Physics **10**, 631–633 (2014).

[13] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd, "Quantum machine learning," Nature **549**, 195–202 (2017).

[14] Vojtech Havlicek, Antonio D. Corcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," Nature **567**, 209 – 212 (2019).

[15] Roman Orus, Samuel Mugel, and Enrique Lizaso, "Quantum computing for finance: Overview and prospects," Reviews in Physics **4**, 100028 (2019).

[16] Patrick Rebentrost and Seth Lloyd, "Quantum computational finance: quantum algorithm for portfolio optimization," (2018), arXiv:1811.03975 .

[17] Stefan Woerner and Daniel J. Egger, "Quantum risk analysis," npj Quantum Information **5**, 15 (2019).

[18] Patrick Rebentrost, Brajesh Gupt, and Thomas R. Bromley, "Quantum computational finance: Monte carlo pricing of financial derivatives," Phys. Rev. A **98**, 022321 (2018).

[19] Christa Zoufal, Aurélien Lucchi, and Stefan Woerner, "Quantum generative adversarial networks for learning and loading random distributions," npj Quantum Information **5**, 1–9 (2019).

[20] Ana Martin, Bruno Candelas, Angel Rodriguez-Rozas, Jose D. Martin-Guerrero, Xi Chen, Lucas Lamata, Roman Orus, Enrique Solano, and Mikel Sanz, "Towards pricing financial derivatives with an ibm quantum computer," (2019), arXiv:1904.05803 .

[21] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp, "Quantum Amplitude Amplification and Estimation," Contemporary Mathematics **305** (2002), 10.1090/conm/305/05215.

[22] Yohichi Suzuki, Shumpei Uno, Rudy Raymond, Tomoki Tanaka, Tamiya Onodera, and Naoki Yamamoto, "Amplitude estimation without phase estimation," Quantum Information Processing **19**, 75 (2020).

[23] Reuven Y. Rubinstein, *Simulation and the Monte Carlo Method*, Wiley Series in Probability and Statistics (Wiley, 1981).

[24] Daniel S Abrams and Colin P Williams, "Fast quantum algorithms for numerical integrals and stochastic processes," (1999), arxiv:quant-ph/9908083 .

[25] Ashley Montanaro, "Quantum speedup of monte carlo methods," Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences **471** (2015), 10.1098/rspa.2015.0301.

[26] A. Yu. Kitaev, "Quantum measurements and the Abelian Stabilizer Problem," (1995), arXiv:quant-ph/9511026 .

[27] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca, "Quantum algorithms revisited," Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences **454**, 339–354 (1998).

[28] Paul Glasserman, Philip Heidelberger, and Perwez Shahabuddin, "Efficient Monte Carlo Methods for Value-at-Risk," in *Mastering Risk*, Vol. 2 (2000) pp. 5–18.

[29] Robert C. Merton, "Theory of rational option pricing," The Bell Journal of Economics and Management Science **4**, 141–183 (1973).

[30] Lov Grover and Terry Rudolph, "Creating superpositions that correspond to efficiently integrable probability distributions," (2002), arXiv:quant-ph/0208112 .

[31] Adriano Koshiyama, Nick Firoozye, and Philip Treleaven, "Generative adversarial networks for financial trading strategies fine-tuning and combination," (2019), arXiv:1901.01751 .

[32] Blanka Horvath, Aitor Muguruza, and Mehdi Tomas, "Deep learning volatility," SSRN Electronic Journal (2019), 10.2139/ssrn.3322085.

[33] Martin Plesch and Časlav Brukner, "Quantum-state preparation with universal gate decompositions," Phys. Rev. A **83**, 032302 (2011).

[34] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems 27*, edited by Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger (Curran Associates, Inc., 2014) pp. 2672–2680.

[35] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. Divincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter, "Elementary gates for quantum computation," Phys. Rev. A **52**, 3457–3467 (1995).

[36] Steven A Cuccaro, Thomas G Draper, Samuel A Kutin, and David Petrie Moulton, "A new quantum ripple-carry addition circuit," (2004), arxiv:quant-ph/0410184 .

[37] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, Jerry M. Chow, Antonio D. Córcoles-Gonzales, Abigail J. Cross, Andrew Cross, Juan Cruz-Benito, Chris Culver, Salvador De La Puente González, Enrique De La Torre, Delton Ding, Eugene Dumitrescu, Ivan Duran, Pieter Eendebak, Mark Everitt, Ismael Faro Sertage, Albert Frisch, Andreas Fuhrer, Jay Gambetta, Borja Godoy Gago, Juan Gomez-Mosquera, Donny Greenberg, Ikko Hamamura, Vojtech Havlicek, Joe Hellmers, Łukasz Herok, Hiroshi Horii, Shaohan Hu, Takashi Imamichi, Toshinari Itoko, Ali Javadi-Abhari, Naoki Kanazawa, Anton Karazeev, Kevin Krsulich, Peng Liu, Yang Luh, Yunho Maeng, Manoel Marques, Francisco Jose Martín-Fernández, Douglas T. McClure, David McKay, Srujan Meesala, Antonio Mezzacapo, Nikolaj Moll, Diego Moreda Rodríguez, Giacomo Nannicini, Paul Nation, Pauline Ollitrault, Lee James O'Riordan, Hanhee Paik, Jesús Pérez, Anna Phan, Marco Pistoia, Viktor Prutyanov, Max Reuter, Julia Rice, Abdón Rodríguez Davila, Raymond Harry Putra Rudy, Mingi Ryu, Ninad Sathaye, Chris Schnabel, Eddie Schoute, Kanav Setia, Yunong Shi, Adenilton Silva, Yukio Siraichi, Seyon Sivarajah, John A. Smolin, Mathias Soeken, Hitomi Takahashi, Ivano Tavernelli, Charles Taylor, Pete Taylour, Kenso Trabing, Matthew Treinish, Wes Turner, Desiree Vogt-Lee, Christophe Vuillot, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, Christopher Wood, Stephen Wood, Stefan Wörner, Ismail Yunus Akhalwaya, and Christa Zoufal, "Qiskit: An open-source framework for quantum computing," (2019).

[38] Vlatko Vedral, Adriano Barenco, and Artur Ekert, "Quantum networks for elementary arithmetic operations," Phys. Rev. A **54**, 147–153 (1996).

[39] Thomas G Draper, "Addition on a Quantum Computer," (2000), arXiv:quant-ph/0008033 .

[40] Thomas G Draper, Samuel A Kutin, Eric M Rains, and Krysta M Svore, "A logarithmic-depth quantum carry-lookahead adder," Quantum Information and Computation **6**, 351–369 (2006), arXiv:quant-ph/0406142 .

[41] Ghasem Tarmast, "Multivariate Log-Normal Distribution," International Statistical Institute Proceedings: 53rd Session, Seoul (2001).

[42] Emmanuel Gobet, "Advanced monte carlo methods for barrier and related exotic options," Handbook of Numerical Analysis, **15**, 497 – 528 (2009).

[43] Pavel V. Shevchenko and Pierre Del Moral, "Valuation of barrier options using sequential monte carlo," Journal of Computational Finance **20**, 107–135 (2014).

[44] M. Žnidarič, O. Giraud, and B. Georgeot, "Optimal number of controlled-not gates to generate a three-qubit state," Physical Review A **77**, 032320 (2008).

[45] Vivek V Shende, Stephen S Bullock, and Igor L Markov, "Synthesis of quantum-logic circuits," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **25**, 1000–1010 (2006).

[46] Raban Iten, Oliver Reardon-Smith, Luca Mondada, Ethan Redmond, Ravjot Singh Kohli, and Roger Colbeck, "Introduction to UniversalQCompiler," (2019), arXiv:1904.01072 .

[47] A. Dewes, F. R. Ong, V. Schmitt, R. Lauro, N. Boulant, P. Bertet, D. Vion, and D. Esteve, "Characterization of a two-transmon processor

with individual single-shot qubit readout," Phys. Rev. Lett. **108**, 057002 (2012).

[48] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta, "Error mitigation for short-depth quantum circuits," Phys. Rev. Lett. **119**, 180509 (2017).

[49] E. F. Dumitrescu, A. J. McCaskey, G. Hagen, G. R. Jansen, T. D. Morris, T. Papenbrock, R. C. Pooser, D. J. Dean, and P. Lougovski, "Cloud quantum computing of an atomic nucleus," Phys. Rev. Lett. **120**, 210501 (2018).

[50] Mark Broadie and Paul Glasserman, "Estimating security price derivatives using simulation," Management Science **42** (1996), 10.1287/mnsc.42.2.269.

[51] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland, "Surface codes: Towards practical large-scale quantum computation," Phys. Rev. A **86**, 032324 (2012).

[52] Miroslav Dobšíček, Göran Johansson, Vitaly Shumeiko, and Göran Wendin, "Arbitrary accuracy iterative quantum phase estimation algorithm using a single ancillary qubit: A two-qubit benchmark," Phys. Rev. A **76**, 030306 (2007).

[53] C. J. O'Loan, "Iterative phase estimation," Journal of Physics A: Mathematical and Theoretical **43** (2010), 10.1088/1751-8113/43/1/015301.

[54] Krysta M Svore, Matthew B Hastings, and Michael Freedman, "Faster phase estimation," Quantum Information & Computation **14**, 306–328 (2014), arXiv:1304.0741 .

[55] S.S. Bullock and I.L. Markov, "Smaller circuits for arbitrary n-qubit diagonal computations," Quantum Information & Computation **4**, 027–047 (2004), arXiv:quant-ph/0303039 .